

UNIT-5

Introduction to Neural Networks: Neurons and Neural Nets, Neural Network Structures for PR Applications, Physical Neural Networks, The Artificial Neural Network Model.

Introduction to Neural Pattern Associators and Matrix Approaches: Neural Network Based Pattern Associators, Matrix Approaches (Linear Associative Mappings) and Examples

Introduction to Neural Networks

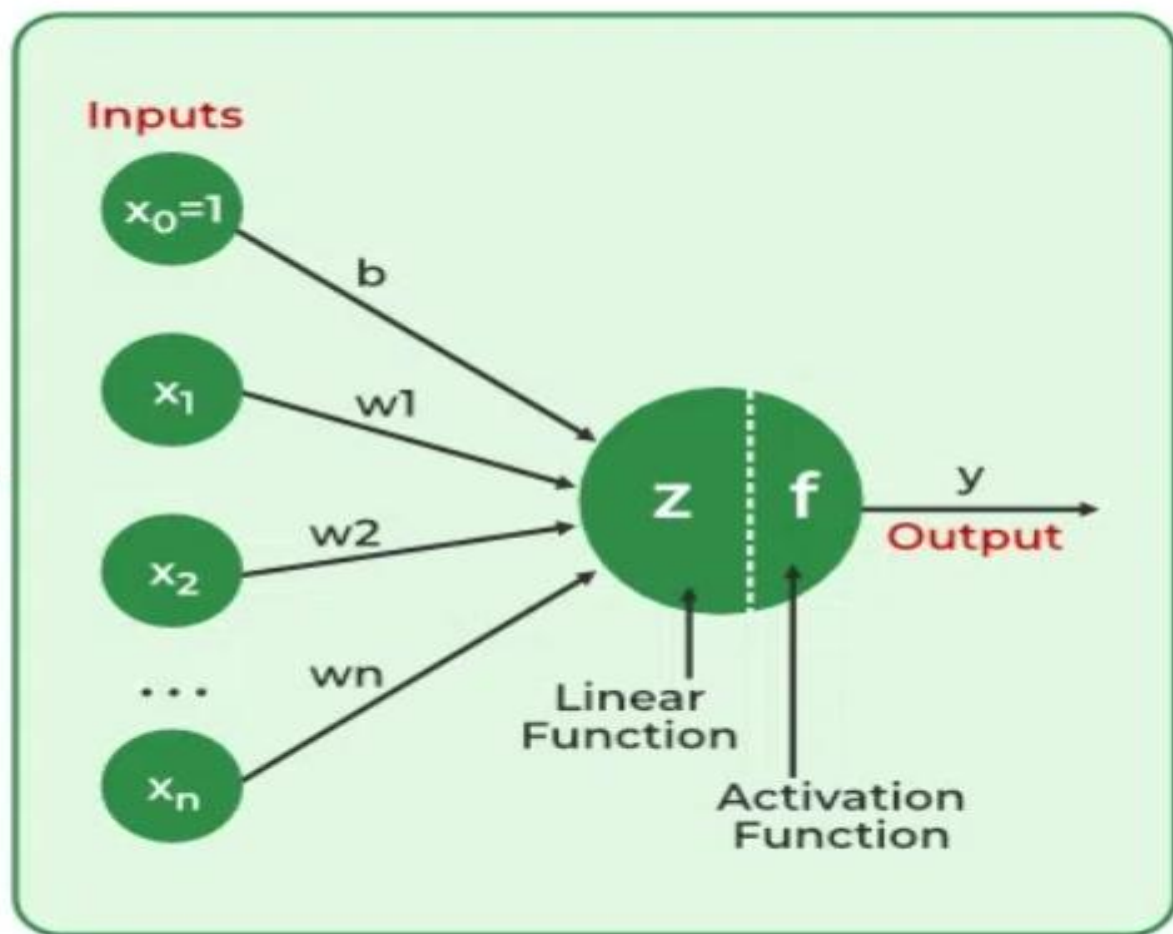
Neural Networks are computational models that mimic the complex functions of the human brain. The neural networks consist of interconnected nodes or neurons that process and learn from data, enabling tasks such as pattern recognition and decision making in machine learning. The article explores more about neural networks, their working, architecture and more.

What are Neural Networks?

Neural networks extract identifying features from data, lacking pre-programmed understanding. Network components include neurons, connections, weights, biases, propagation functions, and a learning rule. Neurons receive inputs, governed by thresholds and activation functions. Connections involve weights and biases regulating information transfer. Learning, adjusting weights and biases, occurs in three stages: input computation, output generation, and iterative refinement enhancing the network's proficiency in diverse tasks.

These include:

1. The neural network is simulated by a new environment.
2. Then the free parameters of the neural network are changed as a result of this simulation.
3. The neural network then responds in a new way to the environment because of the changes in its free parameters.



Importance of Neural Networks

The ability of neural networks to identify patterns, solve intricate puzzles, and adjust to changing surroundings is essential. Their capacity to learn from data has far-reaching effects, ranging from revolutionizing technology like natural language processing and self-driving automobiles to automating decision-making processes and increasing efficiency in numerous industries. The development of artificial intelligence is largely dependent on neural networks, which also drive innovation and influence the direction of technology.

How does Neural Networks work?

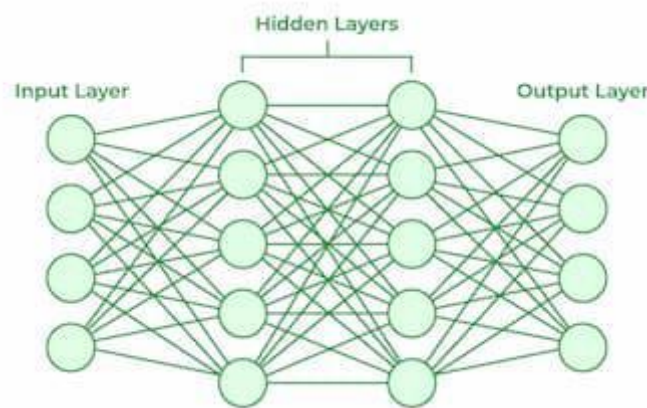
Let's understand with an example of how a neural network works:

Consider a neural network for email classification. The input layer takes features like email content, sender information, and subject. These inputs, multiplied by adjusted weights, pass through hidden layers. The network, through training, learns to

recognize patterns indicating whether an email is spam or not. The output layer, with a binary activation function, predicts whether the email is spam (1) or not (0). As the network iteratively refines its weights through backpropagation, it becomes adept at distinguishing between spam and legitimate emails, showcasing the practicality of neural networks in real-world applications like email filtering.

Working of a Neural Network

Neural networks are complex systems that mimic some features of the functioning of the human brain. It is composed of an input layer, one or more hidden layers, and an output layer made up of layers of artificial neurons that are coupled. The two stages of the basic process are called backpropagation and forward propagation.



Forward Propagation

- **Input Layer:** Each feature in the input layer is represented by a node on the network, which receives input data.
- **Weights and Connections:** The weight of each neuronal connection indicates how strong the connection is. Throughout training, these weights are changed.
- **Hidden Layers:** Each hidden layer neuron processes inputs by multiplying them by weights, adding them up, and then passing them through an activation function. By doing this, non-linearity is introduced, enabling the network to recognize intricate patterns.
- **Output:** The final result is produced by repeating the process until the output layer is reached.
-

Backpropagation

- **Loss Calculation:** The network's output is evaluated against the real goal values, and a loss function is used to compute the difference. For a regression problem, the Mean Squared Error (MSE) is commonly used as the cost function.

Loss Function:

- **Gradient Descent:** Gradient descent is then used by the network to reduce the loss. To lower the inaccuracy, weights are changed based on the derivative of the loss with respect to each weight.
- **Adjusting weights:** The weights are adjusted at each connection by applying this iterative process, or backpropagation, backward across the network.
- **Training:** During training with different data samples, the entire process of forward propagation, loss calculation, and backpropagation is done iteratively, enabling the network to adapt and learn patterns from the data.
- **Activation Functions:** Model non-linearity is introduced by activation functions like the rectified linear unit (ReLU) or sigmoid. Their decision on whether to "fire" a neuron is based on the whole weighted input.

Learning of a Neural Network

1. Learning with supervised learning

In supervised learning, the neural network is guided by a teacher who has access to both input-output pairs. The network creates outputs based on inputs without taking into account the surroundings. By comparing these outputs to the teacher-known desired outputs, an error signal is generated. In order to reduce errors, the network's parameters are changed iteratively and stop when performance is at an acceptable level.

2. Learning with Unsupervised learning

Equivalent output variables are absent in unsupervised learning. Its main goal is to comprehend incoming data's (X) underlying structure. No instructor is present to offer advice. Modeling data patterns and relationships is the intended outcome instead. Words like regression and classification are related to supervised learning, whereas unsupervised learning is associated with clustering and association.

3. Learning with Reinforcement Learning

Through interaction with the environment and feedback in the form of rewards or penalties, the network gains knowledge. Finding a policy or strategy that optimizes cumulative rewards over time is the goal for the network. This kind is frequently utilized in gaming and decision-making applications.

Types of Neural Networks

There are *Five* types of neural networks that can be used.

1. **Feedforward Networks:** A feedforward neural network is a simple artificial neural network architecture in which data moves from input to output in a single direction. It has input, hidden, and output layers; feedback loops are absent. Its straightforward architecture makes it appropriate for a number of applications, such as regression and pattern recognition.

Here are some other characteristics of feedforward neural networks:

- Connections

Each perceptron in one layer is connected to every perceptron on the next layer, but there are no connections between perceptrons in the same layer.

- Training

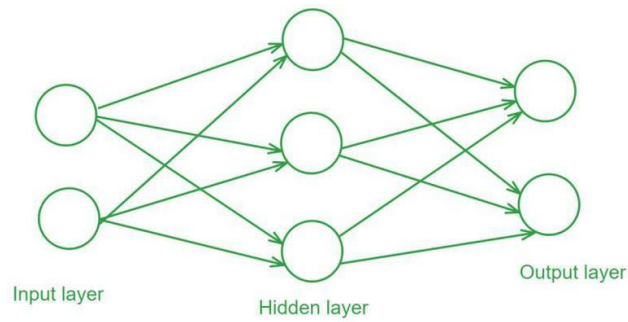
Feedforward neural networks are trained using a process called backpropagation, which adjusts the weights in the network to make better predictions over time.

- Uses

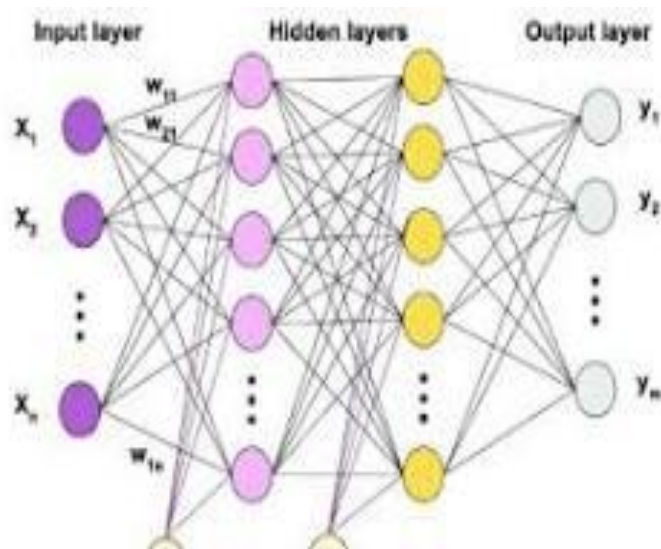
Feedforward neural networks are used for a variety of tasks, including image and voice recognition, natural language processing, and making predictions.

- Limitations

Feedforward neural networks may not be suitable for tasks such as sequential data processing or time series forecasting.

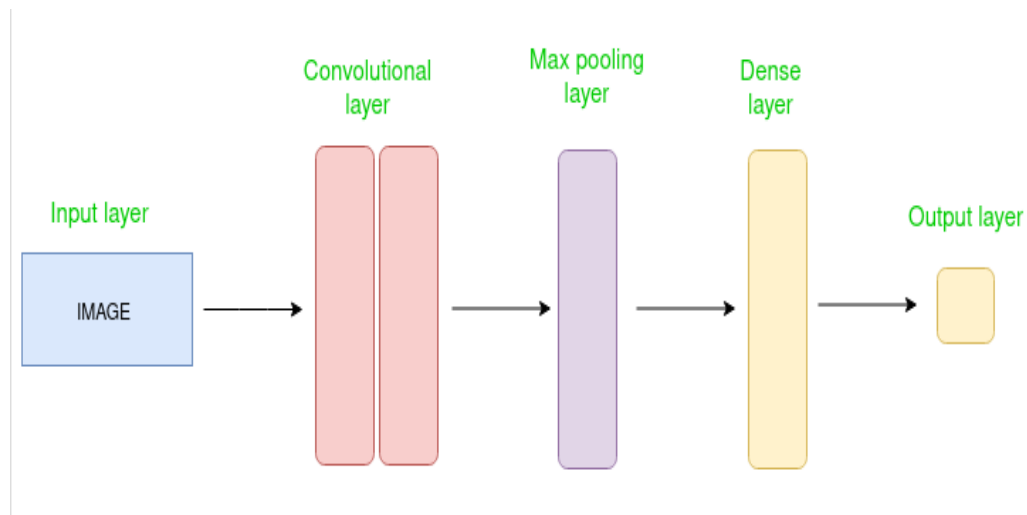


2. **Multilayer Perceptron (MLP):** MLP is a type of feedforward neural network with three or more layers, including an input layer, one or more hidden layers, and an output layer. It uses nonlinear activation functions.



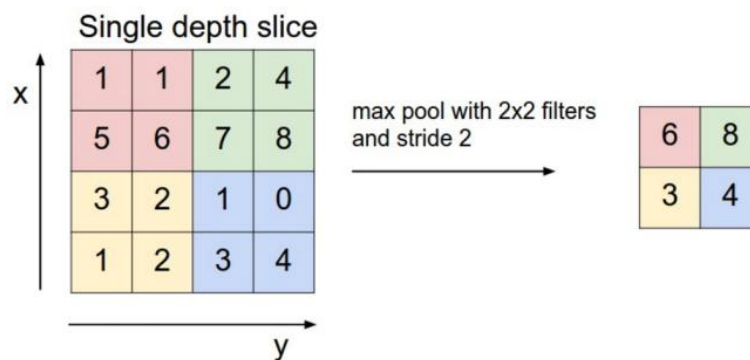
3. **Convolutional Neural Network (CNN):** A Convolutional Neural Network (CNN) is a specialized artificial neural network designed for image processing. It employs convolutional layers to automatically learn hierarchical features from input images, enabling effective image recognition and classification. CNNs have revolutionized computer vision and are pivotal in tasks like object detection and image analysis.

The Convolutional layer applies filters to the input image to extract features, the Pooling layer down samples the image to reduce computation, and the fully connected layer makes the final prediction. The network learns the optimal filters through backpropagation and gradient descent.

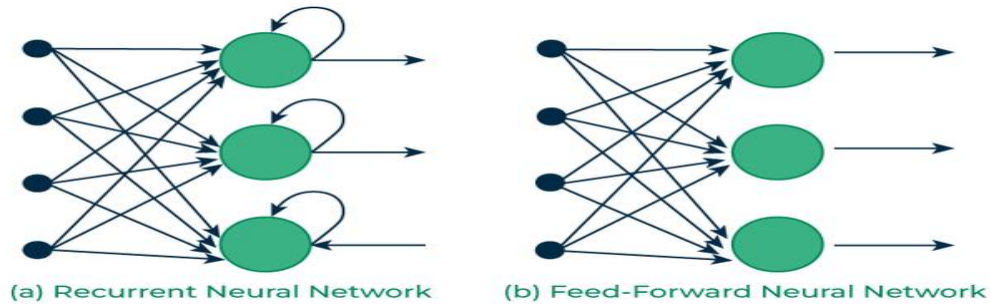


Architecture

- **Flattening:** The resulting feature maps are flattened into a one-dimensional vector after the convolution and pooling layers so they can be passed into a completely linked layer for categorization or regression.
- **Fully Connected Layers:** It takes the input from the previous layer and computes the final classification or regression task.



4. **Recurrent Neural Network (RNN):** An artificial neural network type intended for sequential data processing is called a Recurrent Neural Network (RNN). It is appropriate for applications where contextual dependencies are critical, such as time series prediction and natural language processing, since it makes use of feedback loops, which enable information to survive within the network.



Advantages

1. An RNN remembers each and every piece of information through time. It is useful in time series prediction only because of the feature to remember previous inputs as well. This is called Long Short Term Memory.
2. Recurrent neural networks are even used with convolutional layers to extend the effective pixel neighborhood.

Disadvantages

1. Gradient vanishing and exploding problems.
2. Training an RNN is a very difficult task.
3. It cannot process very long sequences if using tanh or relu as an activation function.

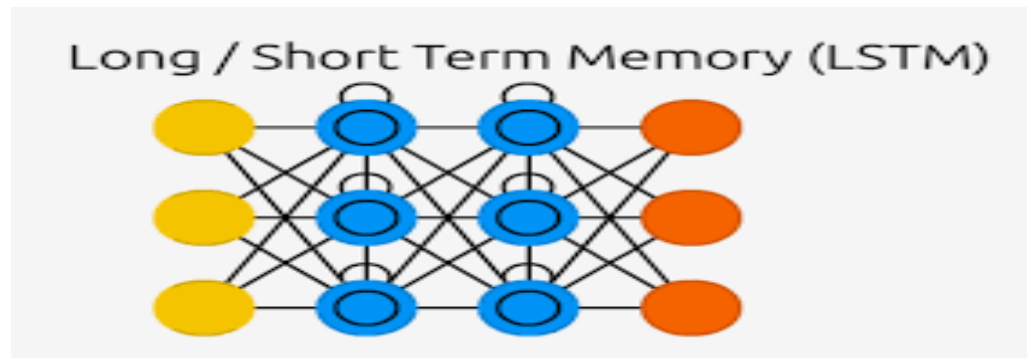
Applications of Recurrent Neural Network

- Language Modelling and Generating Text
 - Speech Recognition
 - Machine Translation
 - Image Recognition, Face detection
 - Time series Forecasting
5. **Long Short-Term Memory (LSTM):** LSTM is a type of RNN that is designed to overcome the vanishing gradient problem in training RNNs. It uses memory cells and gates to selectively read, write, and erase information.

LSTM Architecture

The LSTM architecture involves the memory cell which is controlled by three gates: the input gate, the forget gate, and the output gate. These gates decide what information to add to, remove from, and output from the memory cell.

- The input gate controls what information is added to the memory cell.
- The forget gate controls what information is removed from the memory cell.
- The output gate controls what information is output from the memory cell.



Advantages of Neural Networks

Neural networks are widely used in many different applications because of their many benefits:

- **Adaptability:** Neural networks are useful for activities where the link between inputs and outputs is complex or not well defined because they can adapt to new situations and learn from data.
- **Pattern Recognition:** Their proficiency in pattern recognition renders them efficacious in tasks like as audio and image identification, natural language processing, and other intricate data patterns.
- **Parallel Processing:** Because neural networks are capable of parallel processing by nature, they can process numerous jobs at once, which speeds up and improves the efficiency of computations.
- **Non-Linearity:** Neural networks are able to model and comprehend complicated relationships in data by virtue of the non-linear activation functions found in neurons, which overcome the drawbacks of linear models.

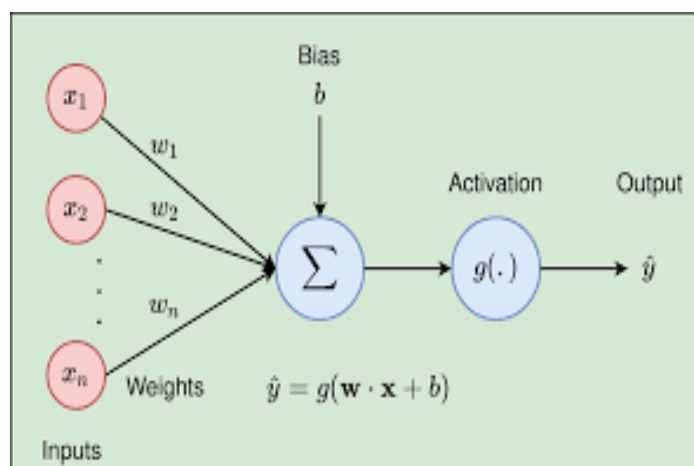
Disadvantages of Neural Networks

Neural networks, while powerful, are not without drawbacks and difficulties:

- **Computational Intensity:** Large neural network training can be a laborious and computationally demanding process that demands a lot of computing power.
- **Black box Nature:** As “black box” models, neural networks pose a problem in important applications since it is difficult to understand how they make decisions.
- **Overfitting:** Overfitting is a phenomenon in which neural networks commit training material to memory rather than identifying patterns in the data. Although regularization approaches help to alleviate this, the problem still exists.
- **Need for Large datasets:** For efficient training, neural networks frequently need sizable, labelled datasets; otherwise, their performance may suffer from incomplete or skewed data.

Neurons and Neural Nets:

Neurons are the building blocks of the human brain, while neural networks are inspired by the brain's structure and function



A neural network is a method in artificial intelligence that teaches computers to process data in a way that is inspired by the human brain. It is a type of machine learning process, called deep learning, that uses interconnected nodes or neurons in a layered structure that resembles the human brain. It creates an adaptive system that computers use to learn from their mistakes and improve continuously. Thus, artificial neural networks attempt to solve complicated problems, like summarizing documents or recognizing faces, with greater accuracy.

Why are neural networks important?

Neural networks can help computers make intelligent decisions with limited human assistance. This is because they can learn and model the relationships between input and output data that are nonlinear and complex. For instance, they can do the following tasks.

Make generalizations and inferences

Neural networks can comprehend unstructured data and make general observations without explicit training. For instance, they can recognize that two different input sentences have a similar meaning:

- Can you tell me how to make the payment?
- How do I transfer money?

A neural network would know that both sentences mean the same thing. Or it would be able to broadly recognize that Baxter Road is a place, but Baxter Smith is a person's name.

What are neural networks used for?

Neural networks have several use cases across many industries, such as the following:

- Medical diagnosis by medical image classification
- Targeted marketing by social network filtering and behavioral data analysis
- Financial predictions by processing historical data of financial instruments
- Electrical load and energy demand forecasting
- Process and quality control
- Chemical compound identification

Important applications of neural networks below----

Computer vision

Computer vision is the ability of computers to extract information and insights from images and videos. With neural networks, computers can distinguish and recognize images similar to humans. Computer vision has several applications, such as the following:

- Visual recognition in self-driving cars so they can recognize road signs and other road users
- Content moderation to automatically remove unsafe or inappropriate content from image and video archives
- Facial recognition to identify faces and recognize attributes like open eyes, glasses, and facial hair
- Image labeling to identify brand logos, clothing, safety gear, and other image details

Speech recognition

Neural networks can analyze human speech despite varying speech patterns, pitch, tone, language, and accent. Virtual assistants like Amazon Alexa and automatic transcription software use speech recognition to do tasks like these:

- Assist call center agents and automatically classify calls
- Convert clinical conversations into documentation in real time
- Accurately subtitle videos and meeting recordings for wider content reach

Natural language processing

Natural language processing (NLP) is the ability to process natural, human-created text. Neural networks help computers gather insights and meaning from text data and documents. NLP has several use cases, including in these functions:

- Automated virtual agents and chatbots
- Automatic organization and classification of written data
- Business intelligence analysis of long-form documents like emails and forms
- Indexing of key phrases that indicate sentiment, like positive and negative comments on social media
- Document summarization and article generation for a given topic

Recommendation Engines:

Neural networks can track user activity to develop personalized recommendations. They can also analyze all user behavior and discover new products or services that interest a specific user. For example, Curalate, a Philadelphia-based startup, helps brands convert social media posts into sales. Brands use Curalate's intelligent product tagging (IPT) service to automate the collection and curation of user-generated social content. IPT uses neural networks to automatically find and recommend products relevant to the user's social media activity.

Neural Network Structures for PR Applications—

1. Feedforward Neural Networks (FNNs)

Application: Basic pattern recognition tasks.

- Structure: Information flows in one direction from input to output.
- Usage: Used for classification problems such as recognizing digits, objects, or faces.

2. Convolutional Neural Networks (CNNs)

Application: Image and video recognition, object detection.

- Structure: Contains convolutional layers, pooling layers, and fully connected layers. Convolutional layers use filters to extract features from images, and pooling layers downsample these features.
- Usage: Highly efficient in image classification, face recognition, and identifying patterns in visual data.

3. Recurrent Neural Networks (RNNs)

Application: Sequence-based data, like speech and text recognition.

- Structure: Includes loops, allowing information to persist over time steps. Handles sequential data where the order of input matters.
- Usage: Common in speech recognition, time series prediction, and handwriting recognition.

4. Long Short-Term Memory Networks (LSTMs)

Application: Long-term sequence data (natural language, time series).

- Structure: A type of RNN with gated cells to manage short-term and long-term dependencies more effectively than regular RNNs.
- Usage: Text classification, speech recognition, and machine translation.

5. Autoencoders

Application: Dimensionality reduction, anomaly detection, and unsupervised pattern recognition.

- Structure: An encoder-decoder structure where input data is compressed into a lower-dimensional representation and then reconstructed.
- Usage: Feature extraction, denoising, and learning representations in an unsupervised manner

6. Generative Adversarial Networks (GANs)

Application: Image generation, data augmentation, and anomaly detection.

- Structure: Two neural networks, a generator and a discriminator, that compete in a game-like framework to generate realistic data.
- Usage: Generating new images, improving data quality in PR tasks by augmenting datasets.

7. Transformers

Application: NLP tasks, machine translation, and sequence-to-sequence learning.

- Structure: Replaces recurrence with attention mechanisms, allowing parallelization over the entire input sequence.
- Usage: Text recognition, document classification, and machine translation.

Physical Neural Networks--

A **physical neural network** is a type of Artificial Neural Network in which an electrically adjustable material is used to emulate the function of a Neural Synapse or a higher-order

(dendritic) neuron model. "Physical" neural network is used to emphasize the reliance on physical hardware used to emulate neurons as opposed to software-based approaches. More generally the term is applicable to other artificial neural networks in which an electrically adjustable resistance material is used to emulate a neural synapse.

OR

Physical Neural Networks can be broadly described as any physical circuit built to emulate the neural connections of a brain.

Why use physical neural networks?

- Energy Considerations drive modern computing advances, and physical neural nets could improve energy efficiencies.
- "Torturing the medium"
- Offers a new paradigm for computing.

Advantages/Disadvantages

Physical Neural Networks (PNNs) are still being developed, so their advantages and disadvantages are commensurate with other developing technologies.

The most important features are:

- "economy of scale"
- Energy savings
- Flexibility in training (both good and bad)
- New paradigm

Application:

- Compilers
- Accessibility
- "conomy of Scale"
- tegration with von Neumann architecture
- Physical implementation of algorithms
- plasticity

Real Time Applications:

Neuromorphic computing is continuing:

- BRAIN initiative proposed by Obama administration
- True North and Loihi
- Medical/Computer Science interest
- DARPA/DOD/DOE

The Artificial Neural Network Model-

Artificial Neural Networks contain artificial neurons which are called units . These units are arranged in a series of layers that together constitute the whole Artificial Neural Network in a system. A layer can have only a dozen units or millions of units as this depends on how the complex neural networks will be required to learn the hidden patterns in the dataset. Commonly, Artificial Neural Network has an input layer, an output layer as well as hidden layers. The input layer receives data from the outside world which the neural network needs to analyze or learn about. Then this data passes through one or multiple hidden layers that transform the input into data that is valuable for the output layer. Finally, the output layer provides an output in the form of a response of the Artificial Neural Networks to input data provided.

In the majority of neural networks, units are interconnected from one layer to another. Each of these connections has weights that determine the influence of one unit on another unit. As the data transfers from one unit to another, the neural network learns more and more about the data which eventually results in an output from the output layer.

Introduction to Neural Pattern Associators and Matrix Approaches:

Pattern Association-

Associative memory neural nets are single-layer nets in which the Weights are determined in such a way that the net can store a set of Pattern associations.

- Each association is an input-output vector pair, $s: t$.
- If each vector t is the same as the vectors with which it is associated, Then the net is called an auto associative memory.
- If the t 's are different from the s 's, the net is called a hetero associative Memory.
- In each of these cases, the net not only learns the specific pattern pairs That were used for training, but also is able to recall the desired response Pattern when given an input stimulus that is similar, but not identical, to The training input.

Before training an associative memory neural net, the original patterns Must be converted to an appropriate representation for computation.

In a simple example, the original pattern Association

Associative memory neural nets are single-layer nets in which the Weights are determined in such a way that the net can store a set of pattern associations.

- Each association is an input-output vector pair, $s: t$.
- If each vector t is the same as the vectors with which it is associated, Then the net is called an auto associative memory.

- If the t's are different from the s's, the net is called a hetero associative Memory.

- In each of these cases, the net not only learns the specific pattern pairs That were used for training, but also is able to recall the desired response.

-Pattern when given an input stimulus that is similar, but not identical, to the training input.

Before training an associative memory neural net, the original patterns Must be converted to an appropriate representation for computation.

In a simple example, the original pattern might consist of "on" and

"Off" signals, and the conversion could be "on" = (+1), "off" = (0)

(Binary representation) or "on" = (+1), "off" = (-1) (bipolar Representation).

TRAINING ALGORITHMS FOR PATTERN ASSOCIATION

1- Hebb Rule for Pattern Association:

- The Hebb rule is the simplest and most common method of Determining the weights for an associative memory neural net.

- We denote our training vector pairs (input training-target output Vectors) as s: t. we then denote our testing input vector as x, which May or may not be the same as one of the training input vectors.

- In the training algorithm of hebb rule the weights initially adjusted to 0, then updated using the following **formula**:

Where,

$$x_i = s_i$$

$$y_j = t_j$$

1. $w_{ij}(\text{new})$:
 - This is the updated value of the weight w_{ij} after the adjustment has been made. It represents the new value of the weight associated with the connection between the i -th input and the j -th output (or neuron).
2. $w_{ij}(\text{old})$:
 - This is the current value of the weight before the update. The weight represents the strength of the connection between the i -th input and the j -th output.
3. η (or ξ , depending on the notation):
 - This is the learning rate, a positive scalar that controls the magnitude of the weight update. It determines how big each adjustment to the weight will be. The learning rate is important in controlling how fast the model learns.
4. x_i :
 - This represents the i -th component of the input vector $\mathbf{x} = (x_1, x_2, \dots, x_n)$. These are the inputs to the model (or the neural network), often corresponding to features in supervised learning problems.
5. y_j :
 - This represents the j -th component of the output vector $\mathbf{y} = (y_1, y_2, \dots, y_m)$. These can be the outputs from neurons or the target outputs in supervised learning (depending on the context). For example, if you are performing a classification task, y_j could be the activation value of the j -th output neuron.
6. $i = 1, \dots, n$:
 - This indicates that i indexes over the n input features or neurons in the network.
7. $j = 1, \dots, m$:
 - This indicates that j indexes over the m output units or neurons in the network.

Delta Rule for Pattern Association

In its original form, the delta rule assumed that the activation function for the output unit was the identity function. Thus, using y for the computed Output for the input vector x , we have

$$y_J = \text{net}_J = \sum_{i=1} x_i w_{iJ}$$

The weights can be updated using the following equation:

$$\Delta w_{ij} = \alpha (t_j - y_j) x_i$$

A simple extension allows for the use of any differentiable activation Function; we shall call this the extended delta rule. The update for the Weight from the I 'th input unit to the J 'th **output unit** is:

$$\Delta w_{IJ} = \alpha (t_J - y_J) x_I f'(\text{net}_J)$$

Meaning:

1. Δw_{IJ} :

- This represents the **change in the weight** between the I -th input and the J -th output (or neuron).
- The **weight** w_{IJ} is the strength of the connection between input unit I and output unit J , and Δw_{IJ} tells us how much to adjust this weight in order to improve the model's performance.

2. α :

- This is the **learning rate** parameter, typically a small positive constant (like 0.01 or 0.1).
- The learning rate controls how large the update to the weights will be during training. If it's too high, the model might overshoot the optimal weights; if it's too low, learning might be very slow.

3. t_J :

- This is the **target output** (also referred to as the desired output) for the J -th neuron or output unit.
- In supervised learning, the target output t_J is the correct label or value that the network should produce for a given input.

4. y_J :

- This is the **actual output** of the J -th neuron or output unit, produced by the network after passing the input through the network's weights and activation function.
- The goal of training is to minimize the difference between t_J (the target) and y_J (the actual output).

5. x_I :

- This is the **input** to the I -th neuron.
- It represents the input value corresponding to the I -th feature or the input layer neuron in the network.

6. $f'(net_J)$:

- This represents the **derivative of the activation function** applied to the J -th neuron's net input (denoted as net_J).
- net_J is the weighted sum of the inputs to the J -th neuron: $net_J = \sum_I w_{IJ}x_I$.
- $f(net_J)$ is the activation function (e.g., sigmoid, ReLU, tanh), and $f'(net_J)$ is its derivative, which is used in backpropagation to compute how the output of the neuron changes in response to changes in the inputs and weights.
- For example, for the **sigmoid function** $f(z) = \frac{1}{1+e^{-z}}$, its derivative is $f'(z) = f(z) \times (1 - f(z))$.

In first rule-

This update rule is a simple form of weight adjustment, where weights are updated based on the product of inputs and outputs, modulated by a learning rate. It can be used in both supervised and unsupervised learning scenarios, and the form of this update is central to many machine learning algorithms. In Hebbian learning, it's a way to strengthen the connection between neurons that are co-activated, while in supervised learning, it's a step toward improving predictions.

In second rule-

It is a key component of the back propagation algorithm used to train neural networks. It adjusts the weights by considering the error between the target and actual output, the input values, and the derivative of the activation function to ensure the network learns effectively and minimizes the error over time.

Matrix Approaches (Linear Associative Mapping) and Example:

In the context of **neural networks** and **pattern associators**, matrix approaches are frequently used to handle **linear transformations**. These transformations play a crucial role in the process of learning, especially in tasks where a network learns to map inputs (patterns) to corresponding outputs (targets). In this section, we will explore how matrix methods are used in **neural pattern associators**, particularly focusing on **linear associative mappings**.

1. Introduction to Neural Pattern Associators

Neural **pattern associators** are a type of **artificial neural network** that learn to associate patterns (input vectors) with specific target patterns (output vectors). The goal of a neural pattern associator is to map each input pattern to a corresponding output pattern.

In the simplest case, a **linear associator** is a network where the relationship between input patterns and output patterns can be described by a linear transformation. This transformation is typically represented by a matrix. The network learns the weights of this matrix such that the input patterns are mapped to the correct output patterns.

2. The Linear Associative Mapping Model

A linear associator maps input vectors $\mathbf{x}$ from an n -dimensional input space to output vectors $\mathbf{y}$ in an m -dimensional output space. The mapping can be represented as:

$$\mathbf{y} = \mathbf{A} \cdot \mathbf{x}$$

Where:

- $\mathbf{y}$ is the output vector.
- $\mathbf{A}$ is the $m \times n$ matrix representing the linear transformation (weights between the input and output).
- $\mathbf{x}$ is the input vector.

The matrix $\mathbf{A}$ encodes the **weights** of the connections between the input and output neurons.

3. Training the Linear Associator

Training a neural pattern associator typically involves learning the weights of the matrix A . The simplest way to train a linear associator is by using **supervised learning**, where the network is provided with pairs of input-output vectors and adjusts the matrix A to minimize the error between the predicted output $\mathbf{y}$ and the target output $\mathbf{t}$.

For a set of training pairs $\{(\mathbf{x}_i, \mathbf{t}_i)\}$, where $i = 1, 2, \dots, N$, the task is to find a matrix A such that:

$$\mathbf{y}_i = A \cdot \mathbf{x}_i \approx \mathbf{t}_i$$

This can be solved using a **least-squares** approach, aiming to minimize the squared error between the predicted and target outputs.

4. Learning Rule for the Linear Associator

The weight matrix A can be learned using a few different methods, such as **gradient descent** or more directly via the **pseudo-inverse** method. The most straightforward way to solve for A is through the **least squares solution**. The least squares method minimizes the following error function:

$$E = \sum_{i=1}^N \|\mathbf{y}_i - \mathbf{t}_i\|^2 = \sum_{i=1}^N \|A \cdot \mathbf{x}_i - \mathbf{t}_i\|^2$$

↓

The solution to this optimization problem is given by:

$$A = (X^T X)^{-1} X^T T$$

Where:

- X is the matrix of input vectors stacked as columns (each $\mathbf{x}_i$ is a column vector).
- T is the matrix of target vectors stacked as columns (each $\mathbf{t}_i$ is a column vector).
- X^T is the transpose of X .

This approach assumes that the input vectors $\mathbf{x}_i$ are linearly independent and that $X^T X$ is invertible. If this is not the case (i.e., the system is over-determined or under-determined), the **pseudo-inverse** can be used as a general solution.

$$A = X^+ T$$

Where X^+ denotes the **Moore-Penrose pseudo-inverse** of X .

Example:

Example 1: Binary Pattern Associator

Let's say you want to create a neural pattern associator that learns to map binary input patterns to corresponding output patterns. Assume we have two input-output pairs:

- $\mathbf{x}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \mathbf{t}_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$
- $\mathbf{x}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \mathbf{t}_2 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$

These pairs are examples of the XOR problem, where each input pattern maps to a different output. We aim to find a weight matrix A that maps these inputs to the desired outputs.

The matrix X containing the input vectors is:

$$X = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$



The matrix T containing the target output vectors is:

$$T = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

The matrix T containing the target output vectors is:

$$T = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Now, we can solve for A using the least squares formula:

$$A = (X^T X)^{-1} X^T T$$

In this case, since $X^T X = I$ (the identity matrix), the solution simplifies to:

$$A = X^T T$$

After performing the matrix multiplication, we obtain:

$$A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Thus, the weight matrix A maps each input vector $\mathbf{x}_1$ and $\mathbf{x}_2$ to the correct output vectors $\mathbf{t}_1$ and $\mathbf{t}_2$.
